Topological Data Analysis Distance Functions: Theory vs. Application

Anirudh Indraganti

March 2025

1 Introduction

This is the final report of the research conducted with Professor Siddharth Vishwanath in the Winter quarter of 2025. The work explores the field of topological data analysis and writing algorithms for better results and efficiency. The work done first began with understanding the basics of the field of topology with a focus on gaining a general understanding of concepts related to persistent homology and generation of persistence diagrams. It then progressed to Julia basics and how to use it for persistent homology. From here, implementation and testing of distance functions was done and compared against their theoretical time complexities. Once this testing was finished, parallel approaches were explored and implemented to improve the pure computational time of the RobustTDA.jl algorithms.

1.1 General Overview

The goal of topological data analysis to find major structures that exist within the shape of a set of data. This information is computed and displayed by persistence diagrams. A persistence diagram is the $2D$ graphical representation of the persistent homology of a dataset. It tracks when topological features are born and when they die. Significant topological features have a long lifetime, which is calculated as $d_i - b_i$ where d_i is the death of a topological feature and b_i is its birth. Below, in Figure 1, we can see an example of creating a topological structure on a set of seven points. The example below uses a Cech complex, where the structure of the data is based on the intersection of radii around each data point, to create this topological structure.

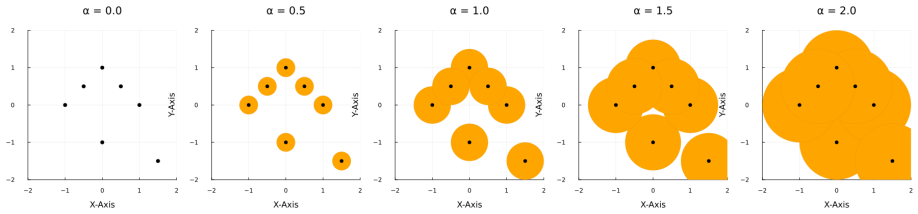


Figure 1: Example Dataset Persistent Homology

In the image above, the radius of the circles around each data point grow in the manner of $[0, 2, 0.5]$. Considering we have data in R^2, we can only have homological features in $H0$ and $H1$, corresponding to the dimensionality of the data. These features for data in R^2 correspond to connected components and holes.

When looking at the image above, we can see that we begin with 7 connected components (one for each point), which then becomes 3 when $r = 1$. These connected components then merge a single one once $r > 1.5$. As for the holes, we see that the presence of holes does not show up until $r = 1.5$, where 2 holes are created, which then disappear once $r = 2$.

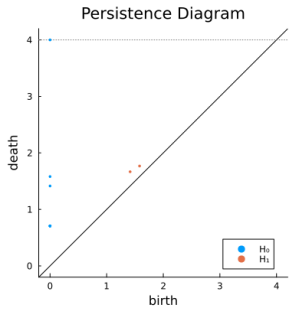


Figure 2: Example Dataset Persistece Diagram

In the image above, the persistence diagram of the dataset from Figure 1 is shown. The information we gained at simply glancing at the data in Figure 1 is generated and displayed in a more concise and precise manner in Figure 2.

From the description above, it is easily noted that the statements were all based on viewing a visualization rather than any true computation. A pipeline to compute this information reliably is as follows:

1. Choose a distance function
2. Create a topological structure for the set of points
3. Create a persistence diagram on this structure

1.2 Distance Functions

Choosing a distance function is the first step in the pipeline. This step directly relates to how the topological structure is formed on the set of points. The reason for this is because the "growing circles" illustration from above are induced by the sub-level sets of the distance function.

There are two major factors that must be analyzed when choosing a distance function: robustness to noise and computational speed. Real world data tends to contain significant noise. Therefore, choosing a distance function that is able to be resistant to noise is crucial for examining important features and how they persist. However, data in the real world can also be very large. This means that we need a distance function that is able to perform calculations at high speeds. Finding a distance function that is able to achieve both of these is critical to the pipeline. Four main functions that will be focused on in this paper are the nearest neighbors (NN where k=1), kernel-density-estimation (KDE with a gaussian kernel), distance-to-measure (DTM), and median-of-means (MoM).

Different distance functions provide different benefits. Therefore, we must start by analyzing the time complexity of each distance function and then take into account its robustness to noise. We also have to account for the distance function being applied to a set of grid points. This is denoted as g in the analysis below. The set of data points is denoted as n.

Starting with the nearest neighbors distance function and its time complexity, we can break it down in the following manner:

1. Construction of a KD-Tree for the dataset: $O(n\log(n))$
2. Finding the nearest neighbor of a single point: $O(\log(n))$
    * Doing this for g number of points: $O(g\log n)$

This simplifies the NN distance function time complexity to be $O(n\log n) + O(g\log n))$. However, the nearest neighbors distance function is not robust to noise. In fact, it struggles mightily with the presence of noise in a dataset. Though this function is a great starting point, it is not so useful in practice.

$$NN_1(x) = \text{argmin}_{y\in X\setminus\{x\}} d(x,y) \text{ where } d(x,y) = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2}$$

The DTM function is next to analyze. Below is a breakdown of the time complexity:

1. KD-Tree construction on dataset: $O(n\log(n))$
2. Find m nearest neighbors for a single evaluation point: $O(m\log(n))$
    * For g grid points: $O(gm\log n)$.

Combined, the time complexity comes out to be $O(n \log n) + O(gm \log n)$. The DTM function also is more resistant to outliers than the nearest-neighbors function. This also has a better time complexity than the KDE function. However, there are limitations to the DTM function. The first is that computing the DTM function effectively is challenging. Since the computations are closely related to high-order Voronoi diagrams, it is computationally expensive to approximate the value of the function. The second is its relation to the amount of neighbors chosen (denoted as m). The value of the DTM function is heavily reliant on this variable. There also is no trivial way to determine an optimal value for m. This makes DTM a function that is not as desireable to work with, even though it has certain benefits that make it better than the previous two distance functions.

$$DTM_m(x) = \left(\frac{1}{m} \sum_{i=1}^{m} d(x, x_{(i)})^r \right)^{\frac{1}{r}}$$

The median-of-means function is the last distance function that will be analyzed. The following is its time complexity breakdown:

1. Shuffle the dataset: $O(n)$
2. Partition dataset into Q folds: $O(n)$
3. Construct KD-Trees for each partition: $O(n \log \frac{n}{Q})$
4. For single point g_i, search each KD-Tree for m nearest-neighbors in each partition: $O(Qm \log \frac{n}{Q})$
5. Find mean of each partition's results: $O(Qm)$
6. Find median of Q means: $O(Q \log Q)$

Combining the time complexities and calculating them for a set of g evaluation points, the time complexity comes out to be:
$O(n) + O(n) + O(n \log \frac{n}{Q}) + g(O(Qm \log nQ) + O(Qm) + O(Q \log Q)) =$
$O(n) + O(n \log \frac{n}{Q}) + O(gQm \log \frac{n}{Q}) =$
$O(n \log \frac{n}{Q}) + O(gQm \log \frac{n}{Q})$

This time complexity seems to be slightly lower than the DTM time complexity. This function also has the added benefit of being more resistant to noise than the DTM function. According to [1], the MoM distance function achieves near minimax-optimal performance in adversarial settings. This algorithm is also simple to make parallel due to the independence of calculations across partitions. This will be explored in terms of time complexity in the next paragraph. There are weaknesses to this function as well. As the same with DTM, the choice of key parameters such as Q and m are vital to the performance of MoM. This may require a variety of methods and cross-validation to ensure good choices are made for these parameters.

In a parallel environment, MoM time complexity can be reworked slightly. Though the pure number of computations are the same, the pure time taken to run them is decreased in theory. We can do so by parallelizing steps 3 and 4.

3. Construct KD-Trees in parallel: $O(\frac{n}{Q} \log \frac{n}{Q})$

4. For single point g_i, search trees in parallel finding m nearest neighbors: $O(m \log \frac{n}{Q})$

Combining the time complexities, we get a new time complexity of $O(\frac{n}{Q} \log \frac{n}{Q}) + O(gm \log \frac{n}{Q})$. This can give a relative idea of how a parallel version of this function can perform.

$$\mu_i = \frac{Q}{n} \sum_{x \in S_i} x \text{ for } i = 1, ..., \text{Q where } S_i \text{ is a partition of the dataset}$$

$$\text{MoM} = \text{median}(\mu_1, \mu_2, \ ..., \ \mu_Q)$$

The image below shows these functions in action with an example set of data of 500 points in R^2. The major topological feature that is desired to be clearly seen is the circle. The dataset samples a random 300 points from a circle of $r = 1$ and has a center at the origin. The rest of the points are uniform noise within the range of the circle's points. The resolution size chosen for this example was 128. For the DTM function, $m = 15$ and $r = 1$ were considered while for MoM, $Q = 20$ and $m = 1$. Keep in mind these images do change significantly with different parameters. Below is merely an example of these distance functions in action.

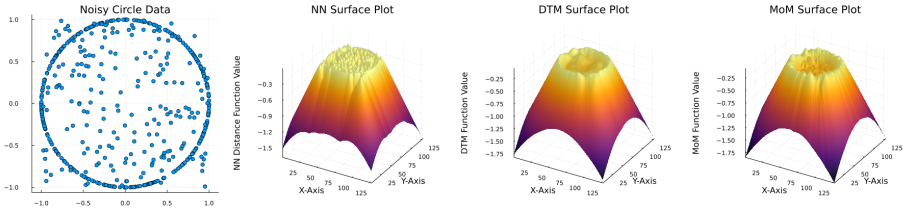


Figure 3: Distance Functions With Noisy Circle Dataset

1.3 Creating Structure On Data

Now that distance functions have been introduced, the actual creation of the topological structure is the next step in the pipeline. One of the major ways that this is done is through the creation of a grid and computing the distance function values on it. These values are then used to create a cubical complex, a type of simplicial complex that creates its simplicies based on cubes rather than triangles such as in other complexes like the Rips or Cech. From here, the persistence diagram is computed, giving us the information we require about the homological features of the data.

The grid used is predefined and usually a square matrix with a designated resolution size, similar to an image on a computer. As a result, the method above assumes uniform importance for all points in the predefined grid when

this is usually not the case. Due to the nature of the grid, it also leads to an exponential increase in the number of cells when finer details are necessary.

The solution to this major issue that is provided in [1] is a weighted CW (Cellular Weak) complex. This method places weights on points that contain more important homological information. Therefore, it allows for resolutions to be adaptive rather than predefined, reducing pre-processing time. With no grid refinement, this also means the construction of a filtration is less expensive, leading to faster computations. With a weighted complex, it still takes in a distance function as input. However, instead of the circles being uniform in radius, as seen in Figure 1, are instead different depending on their importance of information.

Weights can be assigned in different ways where one single way is not standard. These weights can represent local density estimation, a predefined function, or some other application-based criteria.

1.4 Persistence Diagrams

The construction of a persistence diagram is based on a filtration and how this filtration changes under different conditions. As mentioned above, this condition can be based on whatever type of filtration is being constructed. For the example above, a Cech complex was created. The persistence diagrams above tracked the birth and death of important homological features as the condition of the radius of each circle increased. The ability to track these features reliably is where the choice of the distance function comes into play. Below are the persistence diagrams of the distance functions in Figure 3.

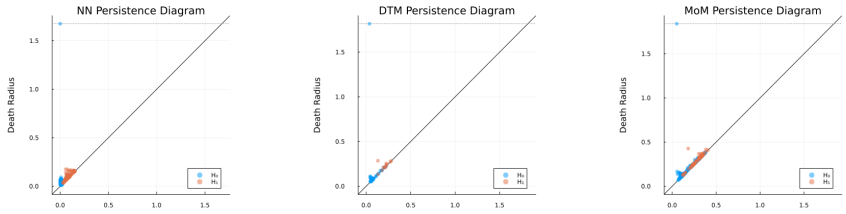


Figure 4: Persistence Diagrams on Distance Functions

The goal was to detect the circle that was present in the dataset. This is a circle, which is an $H1$ feature, meaning the interest is in the red points on the graph. In incremental improvement, it is shown that this circle persisted the least with NN, next with DTM, and most with MoM. The difference is slight, but noticeable. This is mererly an example, but the general principle is to allow major features in the data to persist regardless of noise. As mentioned, DTM and MoM did a better job of that in comparison to NN.

The specific goal of the work done in this paper resides in the analysis of computational runtimes of different distance functions and how to potentially improve upon them. However, the work done prior to this was listed above, creating a general understanding for the purpose of the work being done in the Methodology and subsequent Results sections.

2 Methodology

The first experiments done were to understand the computational cost of increasing the resolution size of a grid in the classical TDA approach. The anticipated result was some form of exponential increase as the resolution increases. In this experiment, the three distance functions mentioned above were tested to visualize the change in computational time increase as grid size increased. To keep the focus on resolution size, the dataset size was kept constant at 50,000 points. The grid sizes ($n \times n$) tested were 16, 32, 48, and 64.

Theoretical time complexities were listed in the Introduction for the three main distance functions. To put this to the test, the next experiment run tested the scaling of these functions as the dataset size increased. The dataset sizes were 10,000, 20,000, 30,000, 40,000, and 50,000. Along with the main distance functions, a parallel form of the MoM function was also tested.

Each distance function's runtimes were measured using the BenchmarkTools.jl package. For the DTM function, 10% of the total number of data points was used as the amount of nearest neighbors m. For the multi-threaded and parallel MoM functions, $Q = 4$, which was the number of available workers on the machine that was used, a Mac Mini while $m = 1$ for the nearest-neighbors to consider.

The theoretical runtimes can be simplified to gain a better understanding of our expected outcome now that certain parameters have been defined. These calculations are listed below:

1. Nearest Neighbors: $O(n \log n) + O(g \log n)$
2. DTM: $O(n \log n) + O(0.1ng \log n)$
3. MoM (Non-Parallel): $O(n \log \frac{n}{4}) + O(4g \log \frac{n}{4})$
4. MoM (Parallel): $O(\frac{n}{4} \log \frac{n}{4}) + O(g \log \frac{n}{4})$

Due to the theoretical runtimes, our order of distance function runtimes should be (from fastest to slowest) Parallel MoM, Nearest Neighbors, Regular MoM, and DTM. However, it is well known that systems do not run in a perfectly parallel environment. Overhead exists, especially when writing the necessary data to each worker. This overhead can be significant, even leading to slower calculations if the task is not sufficiently large. This means testing on large enough data sizes is important to the validity of the experiment. With this understanding, as the data size becomes sufficiently large, the expectation is

that the parallel version of MoM performs better than the serial version. It is then reasoned that with more hidden computations in the MoM function that were mentioned, this "unseen cost" most likely will hinder the performance of the parallel function to be more expensive than the Nearest Neighbors function. Therefore, it is reasoned that the order most likely should come out to be Nearest Neighbors, Parallel MoM, Regular MoM, and lastly DTM.

3 Results

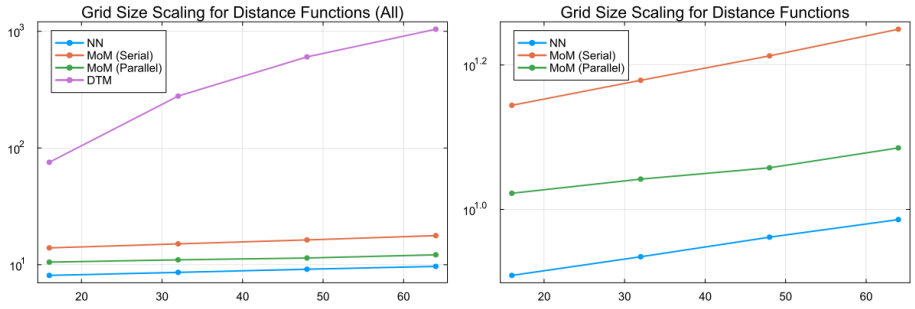


Figure 5: Grid Scaling Runtimes

In the image above, there are two graphs illustrating grid size scaling. The y-axis is $\log_{10}$ millisecond runtime while the x-axis is the grid size. The graph on the left contains all distance functions while the one on the right excludes DTM. The graph on the right does this to gain a better perspective on the functions below DTM. When analyzing these graphs, it is very apparent that the scaling is exponential, as mentioned above. This graph reinforces how expensive grid scaling truly is and how the solution of weighted complexes above can mitigate it.

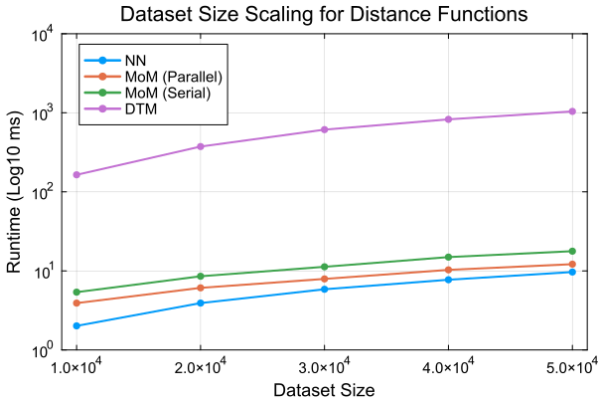


Figure 6: Data Size Scaling Runtimes

The data size scaling graph above confirms the theoretical framework and reasoning mentioned above. The order of runtime from fastest to slowest is the same as mentioned above. This also confirms the implementation of the parallel MoM function was successful. This is also confirmed in Figure 5 with the grid scaling. This also shows that the MoM distance function may be the best out of all choices mentioned in the paper. To go with its theoretical backing in adversarial environments mentioned above, it also has the best runtime theoretically and in application. This evidence backs the work done in [1].

4 Conclusion

Overall, this paper was a basic introduction to the concepts of TDA. There are much more interesting questions and further research to be done in this field. The next steps of this work would revolve around weighted CW complexes, where performance testing and theoretical runtime comparison could be done and cross referenced with this paper.

References

[1] Arxiv. (n.d.). `https://arxiv.org/pdf/2206.01795.pdf`

[2] Hal. (n.d.). `https://hal.science/hal-02093445/file/DTM-filtrations_SoCG.pdf`